

Contact Manager

Project Specification and Functional Requirements

Version 1.0 • March 2026

1. General Information

Project name	Contact Manager
Project creators	Mateusz Piaskowski, Łukasz Sławewski
Programming language	C
Course	Low-level Programming
Instructor	dr inz. Serhii Baraban
Academic year	2025/2026
Form	Individual project or in pairs
Start date	March 17, 2026
Deadline	June 15, 2026

2. Project Description

The goal of this project is to build an interactive Contact Manager application in C. The application allows users to manage a contact list: adding, browsing, editing, searching and deleting records. Data is persistently saved to files so that it is available after restarting the program.

The project meets the requirements for a 5.0 grade, including: modular code structure, dynamic memory management, error handling, file operations, and a graphical user interface.

3. Functional Requirements

3.1 Contact Management

ID	Feature	Description
01	Add contact	User fills in a form with the new contact's data
02	List contacts	Application displays all saved contacts as a list
03	Search	Search by first name, last name or phone number
04	Edit contact	Modify an existing contact record
05	Delete contact	Permanently delete a selected contact with confirmation
06	Sort list	Sort contacts alphabetically by last name

3.2 Contact Data

Each contact contains the following fields (Contact struct):

- First name (char first_name[50])
- Last name (char last_name[50])
- Phone number (char phone[20])
- Email address (char email[100])
- Address (char address[200])
- Note (char note[500])
- Date added (time_t date_added)

3.3 Input Validation

- First name and last name: required, letters only (including special characters)
- Phone number: format +48XXXXXXXXX or 9 digits
- Email address: must contain @ and a domain
- Buffer overflow protection (input length limits enforced)

4. Non-Functional Requirements

- Performance: handle up to 10,000 contacts without noticeable slowdown
- Portability: compiles on Linux (gcc) and Windows (MinGW)
- Memory safety
- Interface: intuitive GUI with mouse support
- Code readability: English comments and function names throughout

5. Modular Architecture

File	Module	Responsibility
main.c	Entry point	Initialization, main loop
logic.c / .h	Application logic	Sorting, searching
io.c / .h	File operations	Save/load CSV and binary
ui.c / .h	User interface	GUI, event handling
validation.c / .h	Validation	Input data checking

6. File Operations

- contacts.csv – main contact database (CSV format)
- contacts.bin – binary backup of data
- app.log – event log (add, edit, delete)
- config.ini – application settings (language, theme, file paths)

On every launch the program automatically loads contacts from file, and saves them when closing.

7. Dynamic Memory Management

Contacts are stored in a dynamic array whose size grows automatically when full.

The ContactArray struct contains:

- Contact *data == pointer to the contacts array
- int size == current number of contacts
- int capacity == number of slots allocated

How it works:

- Initialization: malloc() for initial capacity (e.g. 10 contacts)
- Adding: when size == capacity → realloc() doubles the capacity
- Deleting: shift remaining elements left, decrement size
- Cleanup: free(data) before exiting the program

8. Development Schedule

Milestone	Tasks	Deadline
M1 – Init	Specification, repo structure, Makefile, data structures	April 7, 2026
M2 – Logic	Dynamic memory, file operations	May 5, 2026
M3 – GUI	GUI implementation, error handling, tests	June 11, 2026
Final	Refactoring, documentation, submission on GitHub	June 15, 2026